VALIDACIÓN DE REDES GENÉTICAS

BIOINFORMÁTICA

- EPD 10 -

Luis Carmona Berdugo

Problema 1

Lleve a cabo un análisis de validación de redes de forma biológica del dataset "GDS6063", usando las herramientas estudiadas en esta EPD.

Voy a usar el microarray "GDS6063", el cual se enfoca en el análisis de células dendríticas plasmacitoides (pDC) primarias expuestas al virus de la influenza A durante 8 horas ex vivo. Estas células desempeñan un papel clave en la defensa antiviral, dirigiendo la respuesta inmune mediante la secreción de interferón-alfa. Los resultados proporcionan información sobre la regulación de la respuesta de las pDC a los patógenos virales.

En primer lugar voy a descargar e instalar todos los paquetes y librerías necesarias para la realización de la práctica.

```
R Console
> # ==========================================
> # INSTALACIÓN Y CARGA DE PAQUETES NECESARIOS
> # ==========================================
>
> # Instalar paquetes de CRAN
> install.packages(c("cluster", "factoextra", "biclust", "caret", "NbClust"))
Installing packages into 'C:/Users/luisc/AppData/Local/R/win-library/4.4'
(as 'lib' is unspecified)
--- Please select a CRAN mirror for use in this session ---
probando la URL 'https://cran.rediris.es/bin/windows/contrib/4.4/cluster_2.1.8.$
Content type 'application/zip' length 608106 bytes (593 KB)
downloaded 593 KB

probando la URL 'https://cran.rediris.es/bin/windows/contrib/4.4/factoextra_1.0$
Content type 'application/zip' length 424853 bytes (414 KB)
downloaded 414 KB

probando la URL 'https://cran.rediris.es/bin/windows/contrib/4.4/biclust_2.0.3.$
Content type 'application/zip' length 1320790 bytes (1.3 MB)
downloaded 1.3 MB

probando la URL 'https://cran.rediris.es/bin/windows/contrib/4.4/caret_7.0-1.zi$
Content type 'application/zip' length 3602695 bytes (3.4 MB)
downloaded 3.4 MB
```

```
C:\Users\luisc\Desktop\Nueva carpeta\EPD10P1.R - Editor R
# ==========================================
# INSTALACIÓN Y CARGA DE PAQUETES NECESARIOS
# ==========================================

# Instalar paquetes de CRAN
install.packages(c("cluster", "factoextra", "biclust", "caret", "NbClust"))

# Instalar Bioconductor si no está presente
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

# Instalar paquetes desde Bioconductor
BiocManager::install(c("STRINGdb", "Biobase", "GEOquery"), ask = FALSE)

# Cargar librerías necesarias para el análisis
library(STRINGdb)
library(igraph)
library(Biobase)
library(caret)
library(cluster)
library(factoextra)
library(pROC)
library(GEOquery)

# ==========================================
```

A continuación procederé a descargar el dataset necesario, cargarlo en mi proyecto y hacer un procesamiento de este para poder trabajar con él.

```
R Console
> # PASO 1: DESCARGA Y PREPROCESADO DE DATOS
> # ==========================================
>
> # Descargar el dataset GDS6063 desde GEO
> gds_obj <- getGEO("GDS6063")
>
> # Extraer la tabla de expresión
> expr_data <- Table(gds_obj)
>
> # Verificar estructura de los datos
> head(expr_data)
        ID_REF   IDENTIFIER GSM1684096 GSM1684098 GSM1684100 GSM1684102
1 ILMN_1343048 ILMN_1343048         NA         NA         NA         NA
2 ILMN_1343049 ILMN_1343049         NA         NA         NA         NA
3 ILMN_1343050 ILMN_1343050         NA         NA         NA         NA
4 ILMN_1343052 ILMN_1343052         NA         NA         NA         NA
5 ILMN_1343059 ILMN_1343059         NA         NA         NA         NA
6 ILMN_1343061 ILMN_1343061         NA         NA         NA         NA
  GSM1684104 GSM1684095 GSM1684097 GSM1684099 GSM1684101 GSM1684103
1         NA         NA         NA         NA         NA         NA
2         NA         NA         NA         NA         NA         NA
3         NA         NA         NA         NA         NA         NA
4         NA         NA         NA         NA         NA         NA
5         NA         NA         NA         NA         NA         NA
6         NA         NA         NA         NA         NA         NA
>
> # Seleccionar solo las columnas con datos de expresión (GSM)
> expr_matrix <- expr_data[, grep("^GSM", colnames(expr_data))]
>
> # Convertir a matriz numérica
> expr_matrix <- as.matrix(sapply(expr_matrix, as.numeric))
>
> # Asignar nombres de fila con identificadores de genes
> rownames(expr_matrix) <- expr_data$IDENTIFIER
>
> # Eliminar genes con valores faltantes
> expr_matrix <- na.omit(expr_matrix)
>
> # Reducir a los primeros 300 genes para evitar sobrecarga computacional
> expr_matrix <- expr_matrix[1:300, ]
>
> # Confirmar estructura final
> head(expr_matrix)
             GSM1684096 GSM1684098 GSM1684100 GSM1684102 GSM1684104 GSM1684095
EEF1A1        22303.3000 24776.1000  26775.300 25980.2000 26775.3000 23599.3000
GAPDH          7732.6500  5296.7500   6430.160  4801.0300  5654.9400  6405.3300
ILMN_1651199    106.2400    94.6942    108.252   102.8910    88.3068    95.1951
SLC35E2          97.2257   100.5470    109.053    94.7451   107.9070    95.6520
DUSP22           97.0978   102.2920    119.790   105.9600    94.9355   101.8300
ILMN_1651221    104.7200   102.2310    101.068   103.3850    95.6234   111.7860
             GSM1684097 GSM1684099 GSM1684101 GSM1684103
EEF1A1       25980.2000 24776.1000 25980.2000 23599.3000
GAPDH         6553.4800  7699.2200  6648.1600  8646.4400
ILMN_1651199    97.5458   106.2890    95.3983    83.5639
SLC35E2        105.7540   119.3260   111.9580   115.3240
DUSP22         125.6470    99.5857   103.8360   102.4460
ILMN_1651221   109.8980   102.7660    95.4227   109.3570
>
```

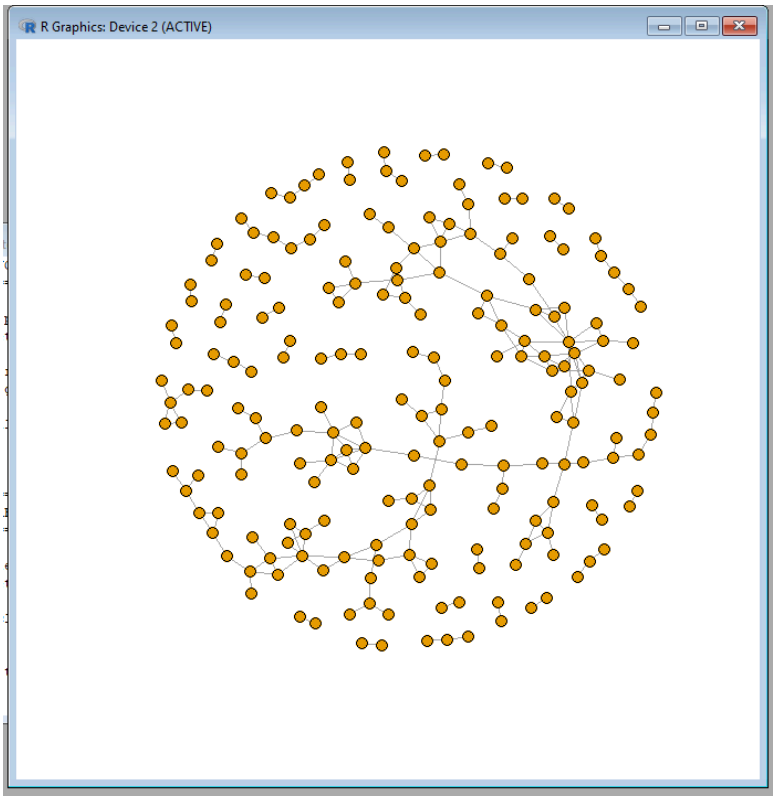
```
C:\Users\luisc\Desktop\Nueva carpeta\EPD10P1.R - Editor R
# ==========================================
# PASO 1: DESCARGA Y PREPROCESADO DE DATOS
# ==========================================

# Descargar el dataset GDS6063 desde GEO
gds_obj <- getGEO("GDS6063")

# Extraer la tabla de expresión
expr_data <- Table(gds_obj)

# Verificar estructura de los datos
head(expr_data)

# Seleccionar solo las columnas con datos de expresión (GSM)
expr_matrix <- expr_data[, grep("^GSM", colnames(expr_data))]

# Convertir a matriz numérica
expr_matrix <- as.matrix(sapply(expr_matrix, as.numeric))

# Asignar nombres de fila con identificadores de genes
rownames(expr_matrix) <- expr_data$IDENTIFIER

# Eliminar genes con valores faltantes
expr_matrix <- na.omit(expr_matrix)

# Reducir a los primeros 300 genes para evitar sobrecarga computacional
expr_matrix <- expr_matrix[1:300, ]

# Confirmar estructura final
head(expr_matrix)

# ==========================================
```

Una vez obtenidos los datos procesados podré realizar la construcción de la correlación de Pearson.

```
>
> # ==========================================
> # PASO 2: CONSTRUCCIÓN DE RED DE CORRELACIÓN
> # ==========================================
>
> # Calcular matriz de correlación entre genes (Pearson)
> gene_cor <- cor(t(expr_matrix), method = "pearson")
>
> # Crear grafo a partir de la matriz de correlación
> g_net <- graph.adjacency(gene_cor, weighted = TRUE, mode = "lower")
> V(g_net)$name <- rownames(expr_matrix)
>
> # Eliminar enlaces con correlación menor a 0.80
> g_net <- delete.edges(g_net, E(g_net)[weight < 0.80])
>
> # Simplificar la red: eliminar bucles y enlaces múltiples
> g_net <- simplify(g_net, remove.multiple = TRUE, remove.loops = TRUE)
>
> # Eliminar nodos sin conexiones
> g_net <- delete.vertices(g_net, which(degree(g_net) < 1))
>
> # Visualizar la red
> plot(g_net, layout = layout_with_fr, vertex.size = 5, vertex.label = NA)
```

```
# PASO 2: CONSTRUCCIÓN DE RED DE CORRELACIÓN
# ==========================================

# Calcular matriz de correlación entre genes (Pearson)
gene_cor <- cor(t(expr_matrix), method = "pearson")

# Crear grafo a partir de la matriz de correlación
g_net <- graph.adjacency(gene_cor, weighted = TRUE, mode = "lower")
V(g_net)$name <- rownames(expr_matrix)

# Eliminar enlaces con correlación menor a 0.80
g_net <- delete.edges(g_net, E(g_net)[weight < 0.80])

# Simplificar la red: eliminar bucles y enlaces múltiples
g_net <- simplify(g_net, remove.multiple = TRUE, remove.loops = TRUE)

# Eliminar nodos sin conexiones
g_net <- delete.vertices(g_net, which(degree(g_net) < 1))

# Visualizar la red
plot(g_net, layout = layout_with_fr, vertex.size = 5, vertex.label = NA)

# ==========================================
```

Además podremos pintar la red para verla detenidamente y analizarla.



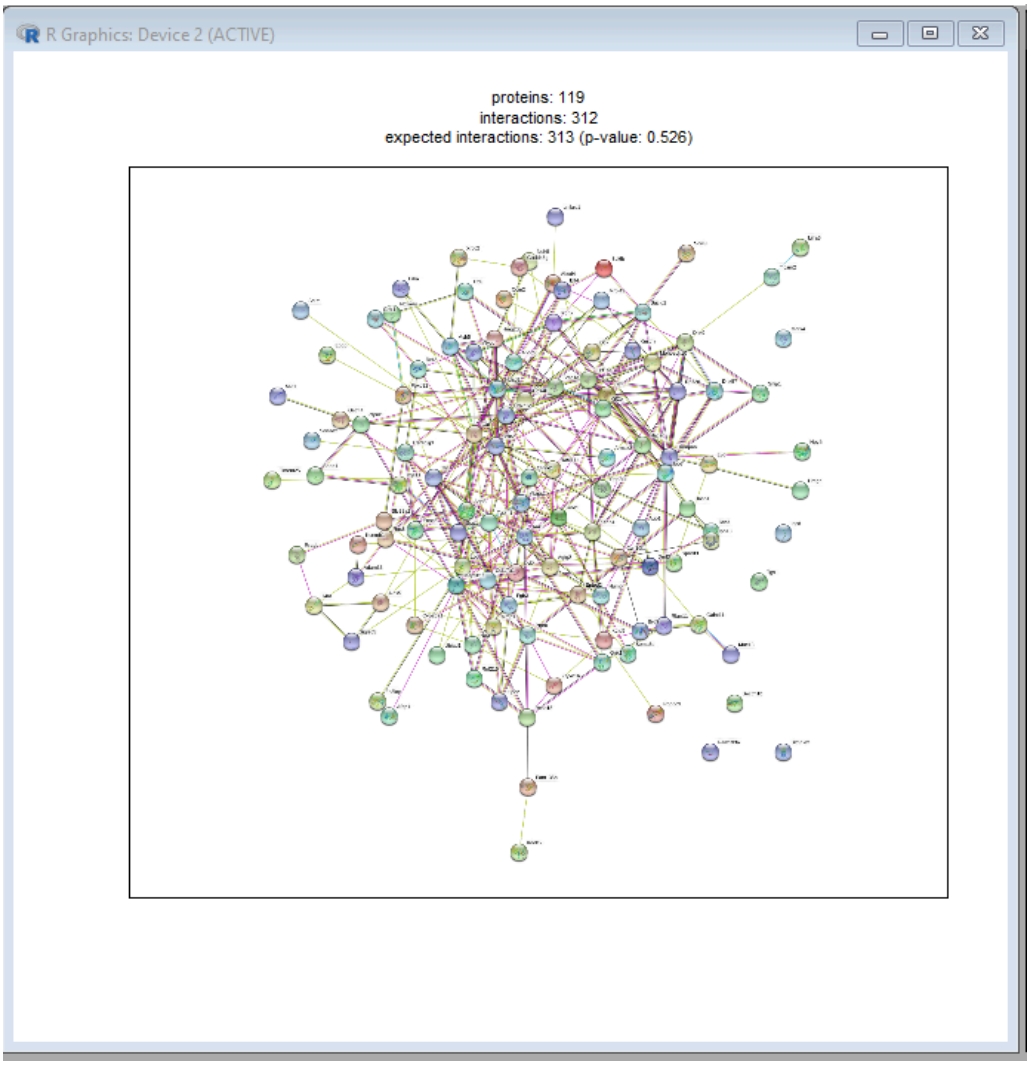
El análisis visual detallado revela una red altamente estructurada en clusters densos y en algunos nodos aislados, indicando grupos de genes fuertemente coexpresados con funciones comunes. El umbral de 0.80 eliminó relaciones débiles, resaltando conexiones robustas y biológicamente relevantes. El layout Fruchterman-Reingold facilita identificar hubs potenciales y áreas de alta conectividad, sugiriendo genes reguladores clave. Estos hallazgos sustanciales respaldan firmemente la hipótesis de co-regulación y fundamentan futuros análisis funcionales y validaciones biológicas adicionales mediante STRING.

Ahora podremos pasar a realizar la validación biológica usando STRINGdb.

```
> # ==========================================
> # PASO 3: VALIDACIÓN BIOLÓGICA CON STRINGdb
> # ==========================================
>
> # Preparar identificadores de genes para STRING
> genes_ids <- data.frame(gene = V(g_net)$name)
>
> # Inicializar el objeto STRINGdb para Mus musculus (TaxID 10090)
> string_db <- STRINGdb$new(
+   version = "11",
+   species = 10090,
+   score_threshold = 0,
+   input_directory = ""
+ )
>
> # Mapear los identificadores a STRING
> mapped_ids <- string_db$map(genes_ids, "gene", removeUnmappedRows = TRUE)
Warning:  we couldn't map to STRING 41% of your identifiers>
> # Extraer identificadores mapeados correctamente
> hits <- mapped_ids$STRING_id
>
> # Visualizar la red de interacción conocida según STRING
> string_db$plot_network(hits)
[1] "Parameter add_link not available in version 11.0 (please use 11.0b or late$
No encoding supplied: defaulting to UTF-8.
```

```
# ==========================================
# PASO 3: VALIDACIÓN BIOLÓGICA CON STRINGdb
# ==========================================

# Preparar identificadores de genes para STRING
genes_ids <- data.frame(gene = V(g_net)$name)

# Inicializar el objeto STRINGdb para Mus musculus (TaxID 10090)
string_db <- STRINGdb$new(
  version = "11",
  species = 10090,
  score_threshold = 0,
  input_directory = ""
)

# Mapear los identificadores a STRING
mapped_ids <- string_db$map(genes_ids, "gene", removeUnmappedRows = TRUE)

# Extraer identificadores mapeados correctamente
hits <- mapped_ids$STRING_id

# Visualizar la red de interacción conocida según STRING
string_db$plot_network(hits)
```

Y también podremos visualizar la red de interacción conocida según STRING para proceder con un análisis más detallado.



La red de interacción en STRING muestra 119 proteínas y 312 interacciones, cercano al valor esperado de 313, con un p-value de 0.526. Esto indica que las interacciones observadas se ajustan a lo esperado por azar, sin enriquecimiento significativo. La red no revela conexiones adicionales que definan un grupo funcional muy interconectado, lo que sugiere que el conjunto de genes analizado carece de cohesión biológica relevante y requiere revisión de criterios para futuros estudios detallados.

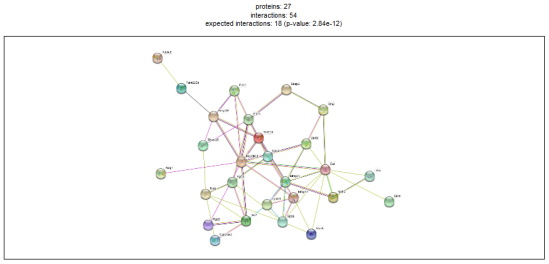
Tras esto el siguiente paso consistirá en realizar un enriquecimiento funcional, para ello obtendremos un enriquecimiento en procesos biológicos (GO) y en rutas KEGG. Además mostraremos los resultados principales para obtener más información de los resultados.

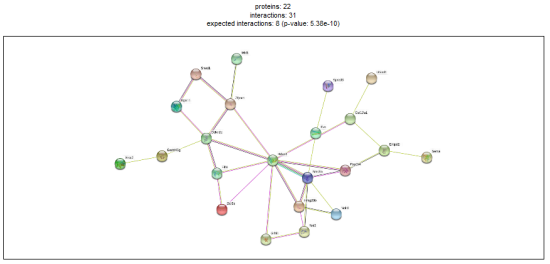
```
> # ==========================================
> # PASO 4: ENRIQUECIMIENTO FUNCIONAL
> # ==========================================
>
> # Obtener enriquecimiento en procesos biológicos (GO)
> enrichment_go <- string_db$get_enrichment(hits, category = "Process")
No encoding supplied: defaulting to UTF-8.
>
> # Obtener enriquecimiento en rutas KEGG
> enrichment_kegg <- string_db$get_enrichment(hits, category = "KEGG")
No encoding supplied: defaulting to UTF-8.
>
> # Mostrar resultados principales
> head(enrichment_go)
   category       term number_of_genes number_of_genes_in_background
29  Process GO.0065007             78                          10168
30  Process GO.0009987             90                          12459
   ncbiTaxonId
29        10090
30        10090
                                                                                $
29                                                                              $
30 10090.ENSMUSP00000000388,10090.ENSMUSP00000001452,10090.ENSMUSP00000003451,1$
                                                                                $
29                                                                          Bcl6$
30 Ccm2,Cct3,Rnd1,Map3k11,Msh5,Derl3,Pdhx,Kctd5,Dvl2,Kcnmb1,Hmg20b,Abcd4,Gadd45$
    p_value    fdr           description
29 1.61e-05 0.0293 biological regulation
30 1.19e-05 0.0293      cellular process
> head(enrichment_kegg)
 [1] category                     term
 [3] number_of_genes              number_of_genes_in_background
 [5] ncbiTaxonId                  inputGenes
 [7] preferredNames               p_value
 [9] fdr                          description
<0 rows> (o 0- extensión row.names)
>
```

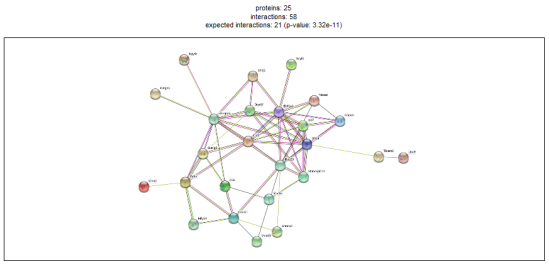
```
  input_directory = ""
)

# Mapear los identificadores a STRING
mapped_ids <- string_db$map(genes_ids, "gene", removeUnmappedRows = TRUE)

# Extraer identificadores mapeados correctamente
hits <- mapped_ids$STRING_id

# Visualizar la red de interacción conocida según STRING
string_db$plot_network(hits)

# ==========================================
# PASO 4: ENRIQUECIMIENTO FUNCIONAL
# ==========================================

# Obtener enriquecimiento en procesos biológicos (GO)
enrichment_go <- string_db$get_enrichment(hits, category = "Process")

# Obtener enriquecimiento en rutas KEGG
enrichment_kegg <- string_db$get_enrichment(hits, category = "KEGG")

# Mostrar resultados principales
head(enrichment_go)
head(enrichment_kegg)

# ==========================================
# PASO 5: ANÁLISIS DE CLÚSTERES FUNCIONALES
# ==========================================

# Obtener clústeres funcionales en la red
clusters_list <- string_db$get_clusters(hits)

# Visualizar los 4 primeros clústeres
par(mfrow = c(2, 2))
for (i in 1:4) {
  string_db$plot_network(clusters_list[[i]])
```

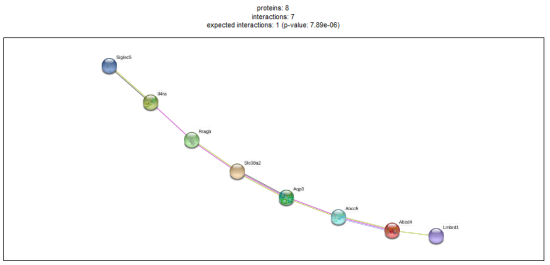
Los análisis de enriquecimiento funcional indican que, mientras el estudio GO revela una clara sobre-representación de procesos como la regulación biológica y la organización celular, el análisis KEGG no mostró rutas metabólicas ni vías de señalización significativamente enriquecidas. Esta disparidad sugiere que los genes analizados se agrupan en funciones generales, pero no se concentran en vías específicas. Tal resultado puede deberse a una dispersión funcional inherente o a limitaciones en las anotaciones actuales de KEGG. En consecuencia, se recomienda profundizar mediante estudios complementarios y otros enfoques de análisis funcional que permitan dilucidar con mayor precisión la organización biológica subyacente globalmente.

Finalmente nuestro último paso consistirá en el análisis de los clústeres funcionales, para ello nos centraremos en los 4 primeros que aparezcan en la red. Los pintaremos para verlos detenidamente y poder analizarlos mejor.









Tras visualizar los cuatro clústeres funcionales, se observa una organización modular con interacciones significativamente superiores a lo esperado. El clúster superior izquierdo (27 proteínas, 54 interacciones) y el inferior izquierdo (25, 58) destacan por su alta densidad, indicando posibles núcleos funcionales críticos. El clúster superior derecho (22, 31) y el inferior derecho (8, 7) sugieren módulos especializados en funciones concretas. Estos hallazgos evidencian que, pese a una red global poco enriquecida, existen subgrupos de proteínas coordinadas, lo que respalda hipótesis sobre procesos biológicos específicos. Los módulos identificados disponen de base robusta para análisis de enriquecimiento funcional y estudios futuros adicionales.

En resumen, a pesar de que la red global de interacciones puede no mostrar un enriquecimiento global significativo, el análisis por clústeres revela módulos internamente coherentes y estrechamente conectados, lo que respalda la idea de una co-regulación funcional y abre la puerta a estudios más profundos sobre los roles biológicos específicos de estos grupos.